

Clardentity AI Platform - MVP Software Requirements Specification

Document status: Build-ready - intended for direct use as a Cursor engineering brief

Prepared for: Cursor-driven implementation **Target build model:** Claude Fable 5 (Cursor agent)

Base model for product: OpenAI GPT-5 family (Responses API)

1. Purpose

Clardentity is a cognitive layer built over OpenAI's APIs that turns raw LLM completions into **validated, mode-aware, citation-backed conversations**, presented through a **generic on-screen avatar** that gestures and changes expression as it "speaks." Every user turn is handled by a cognitive companion the user has explicitly chosen (there is no automatic mode-guessing), grounded in retrieved context (conversation memory + uploaded documents), and - critically - every claim in the response is individually checked against its own evidence and given its own score, not just a single opaque confidence number for the whole message.

2. Objectives

1. Implement four cognitive modes as first-class, independently testable pipelines, **manually selected by the user for every turn** (no auto-detection fallback):
 - **Interactive Knowing Companion (IKC)** - information retrieval / fact lookup
 - **Interactive Thinking Companion (ITC)** - structured, multi-step reasoning
 - **Interactive Decision-making Companion (IDC)** - options analysis + recommendation with tradeoffs
 - **Interactive Learning Companion (ILC)** - knowledge transformation (explain, quiz, summarize-to-learn)
2. Every assistant response carries:
 - Conversation-history awareness (the model always sees relevant prior turns)
 - A **per-claim, per-evidence** validation score (not a single black-box number) rolled up into an overall confidence band (Likely Fact / Plausible / Needs Verification)
 - An explicit, always-visible breakdown of exactly which sources back which statements
3. Present responses through a **generic avatar** that performs mode-appropriate gestures and confidence-appropriate facial expressions while "speaking."
4. Support multimodal input: text, audio, images, documents (PDF/DOCX/TXT).

5. Ship an MVP that a single engineer can build in phases using Cursor, with clear file-level scaffolding.
-

3. Scope

3.1 In-Scope (MVP)

- Authentication (email/password + OAuth), JWT sessions
- Workspaces/Projects (multi-conversation containers)
- Chat UI with streaming responses
- **Generic Avatar Companion** - gestures + facial expressions synced to mode and confidence (Section 8)
- **Mandatory manual cognitive-mode selection** - no auto/default mode; the user must pick one of the four modes before a message can be sent (Section 7.2)
- Multimodal input: text, audio (record + upload), images, documents
- OpenAI Responses API integration (text, vision, audio transcription/TTS)
- Conversation memory (short-term window + long-term summarized memory)
- Document ingestion + RAG (pgvector similarity search)
- Query Optimization Agent (ambiguity detection + query rewriting - **not** mode selection)
- Validation Engine with **per-claim and per-evidence scoring**, Reflection Agent, Verification Agent, and cognitive-distortion screening
- Explicit **Evidence Transparency Panel** on every response (Section 9.5)
- Conversation history search
- Conversation export (Markdown/PDF)
- Basic admin configuration (model params, feature flags, avatar/gesture mapping)

3.2 Out-of-Scope (Post-MVP)

- Video ingestion and real-time video reasoning
 - Collaborative/multi-user live sessions
 - Enterprise SSO (SAML/SCIM)
 - Fine-grained RBAC beyond owner/member
 - Analytics dashboards
 - Third-party plugin/tool marketplace
 - 3D / photorealistic avatar, full viseme lip-sync, custom/user-uploaded avatar likenesses (MVP avatar is a single generic, non-identifiable character - see Section 8.5)
-

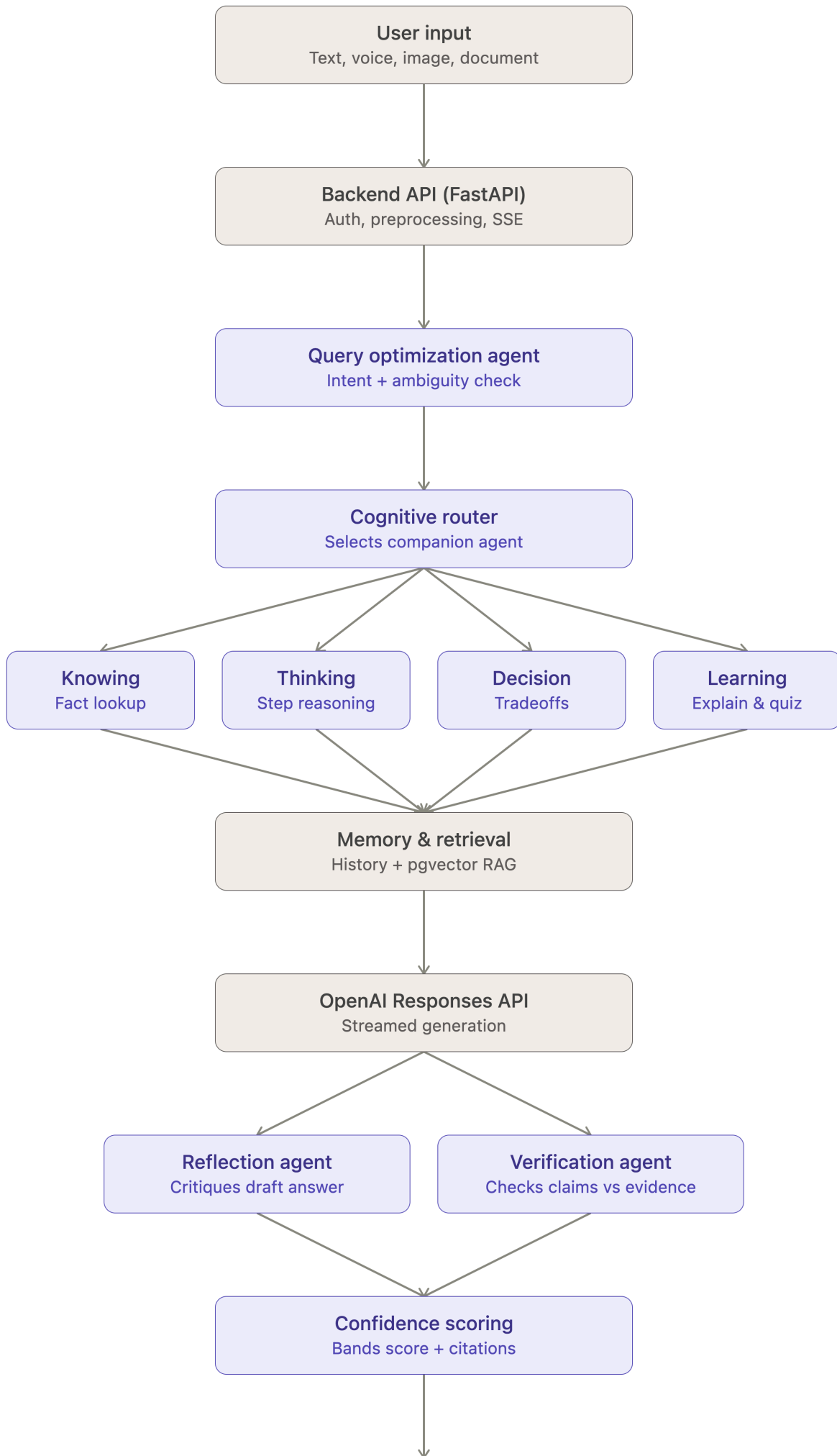
4. Personas & Core Use Cases

Persona	Need	Primary Mode
Analyst	"What does clause 4.2 of this contract say?"	Knowing Companion
Product Manager	"Walk me through the tradeoffs of build vs buy for X"	Thinking / Decision Companion
Student/Researcher	"Turn this 40-page paper into a study guide"	Learning Companion
Ops lead	"Should we renew Vendor A or switch to Vendor B given these three docs?"	Decision Companion

Each use case exercises: document upload → **user selects mode** → retrieval → per-claim validation → evidence-tagged response → avatar reaction.

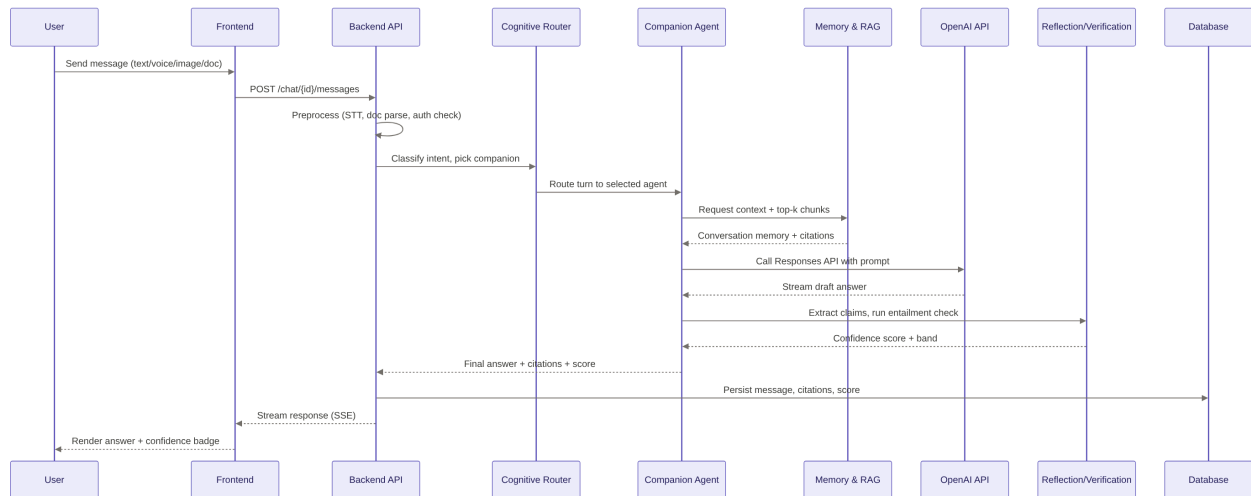
5. System Architecture

5.1 Component & Sequence Diagrams



Response & persist
Stream to client, save to DB

Gray = infrastructure & OpenAI · Purple = cognitive agents



Reconciliation note (read this before implementing the router): the diagrams' "Query optimization agent - Intent + ambiguity check" and "Classify intent, pick companion" labels predate the decision in this version to remove automatic mode detection. As built:

- The **Query Optimization Agent** step still runs, but its job is limited to ambiguity detection and query rewriting/expansion for better retrieval - it never chooses or overrides the cognitive mode.
- The **Cognitive Router** step no longer classifies intent to pick a companion. It simply **validates the mode the user explicitly selected in the UI** and loads that companion's prompt template. If no mode is present on the request, the router rejects it (see Section 7.2).
- Everything downstream of the router (Companion Agent → Memory & Retrieval → OpenAI Responses API → Reflection Agent / Verification Agent → Confidence Scoring → Response & Persist) is unchanged from the diagrams.
- The final "Response & Persist" step now also computes and attaches the **avatar cue** (Section 8) before streaming to the client.

5.2 End-to-End Workflow (expanded)

1. **Input** - text / audio / image / document arrives via `/chat` or `/documents`, **tagged with the user's explicitly selected cognitive mode**.
2. **Preprocessing** - audio → transcript (OpenAI STT); image → vision-ready payload; document → text extraction + chunking.
3. **Query Optimization** - detects ambiguous/underspecified phrasing and may rewrite/expand the query for retrieval; never alters the selected mode.

4. **Cognitive Router** - validates the selected mode against the four allowed values, rejects the request if missing, and loads the corresponding prompt template + system instructions.
 5. **Context Retrieval** - pulls (a) recent conversation window, (b) long-term memory summary, (c) top-k pgvector chunks from workspace documents relevant to the (optimized) query.
 6. **Prompt Builder** - assembles a structured prompt: system instructions (mode-specific, optionally lensed per Section 7.5) + retrieved context + claim-tagging instructions + user turn.
 7. **OpenAI Responses API call** - streamed generation, with the model instructed to tag each discrete claim so it can be individually validated.
 8. **Reflection Agent** - critiques the draft answer for internal consistency, completeness, and tone fit for the selected mode; may trigger a single revision pass.
 9. **Verification Agent** - extracts each tagged claim, matches it against retrieved evidence, runs an entailment check per claim **per piece of evidence**, and screens for wishful/magical-thinking reasoning patterns (Section 9.4).
 10. **Confidence Scoring** - computes a score and band per claim, then rolls those up into a message-level band: `>=90 Likely Fact`, `70-89 Plausible`, `<70 Needs Verification` (Section 9.3).
 11. **Avatar Cue Computation** - derives an `{expression, gesture}` pair from `mode_used` + confidence band + Reflection Agent tone signal + any distortion flag (Section 8).
 12. **Final Response** - streamed to client with inline per-claim citation markers, an always-visible evidence breakdown, a confidence badge, and the avatar cue.
 13. **Persist** - message, per-claim scores, per-evidence scores, avatar cue, token usage, mode used → Postgres; embeddings for any new content → pgvector.
-

6. Functional Requirements (expanded, testable)

ID	Requirement	Acceptance Test
FR1	User can register/login via email+password or OAuth (Google)	New user completes signup → receives JWT → accesses <code>/chat</code>
FR2	User can create/rename/delete Workspaces; each Workspace has its own documents + conversations	Create workspace → upload doc → doc not visible in a different workspace
FR3	Conversations persist across sessions with full turn history	Reload app → prior conversation and messages still present
FR4	Upload PDF/DOCX/TXT/PNG/JPG up to configurable size limit (default 25MB)	Upload each type → status <code>processed</code> within SLA

ID	Requirement	Acceptance Test
FR5	Record or upload audio; transcribed via OpenAI STT before entering pipeline	Upload .mp3 → transcript appears as user message within 10s for <2min clip
FR6	Chat accepts mixed input (text + attached image/doc in same turn)	Send text + image → both included in model prompt
FR7	User must explicitly select one of the four cognitive modes before a message can be sent; there is no automatic mode inference or default. Mode can be changed per turn.	Send button stays disabled until a mode is selected; sending without a mode returns a 400 from /chat
FR8	RAG retrieves top-k relevant chunks from workspace documents before generation	Query about uploaded doc content returns correct citation to that doc
FR9	Verification Agent cross-checks each individual claim against retrieved evidence	A fabricated claim not supported by any evidence is flagged Unsupported at the claim level, dragging the message toward Needs Verification
FR10	Every response displays a message-level confidence band rolled up from individual claim scores (Section 9.3)	Confidence badge visible on every assistant message
FR11	Citations link back to source document + page/chunk, at the individual claim level	Clicking a citation next to a specific sentence opens that exact source excerpt
FR12	Full-text search across conversation history within a workspace	Search term returns matching messages ranked by relevance
FR13	Export a conversation as Markdown or PDF, including per-claim evidence and scores	Export produces a file with all turns, per-claim citations, and confidence bands
FR14	Admin panel: model selection, temperature, top-k retrieval, feature flags, avatar gesture/expression mapping	Changing top-k in admin affects retrieval results on next query
FR15	Every assistant message is accompanied by an avatar performing a gesture and facial expression consistent with the selected mode and the resulting confidence band	Sending a Decision-mode query renders the "weighing options" gesture; a Needs Verification response renders a cautious expression

ID	Requirement	Acceptance Test
FR16	Every claim in a response is individually scored, and every piece of evidence used for that claim is individually scored - not just one score for the whole message	/validation/{message_id} returns a <code>claims[]</code> array, each with its own <code>evidence[]</code> array and per-item scores
FR17	Every response renders an explicit Evidence panel stating, per claim, exactly which source(s) were used (or that none were found)	Expanding "Evidence" under any assistant message shows each sentence/claim mapped to its cited source excerpt(s) and score, with unsupported claims explicitly labeled

7. Cognitive Mode Router - Mode Design

7.1 Mode Definitions

Mode	Purpose	System Prompt Emphasis	Typical Output Shape	Default Avatar Gesture
Knowing	Retrieve and state facts	Precision, brevity, cite sources	Direct answer + citation	Presenting / pointing gesture
Thinking	Structured reasoning	Step-by-step, show intermediate logic	Numbered reasoning chain → conclusion	Chin-stroke / counting-on-fingers gesture
Decision	Compare options, recommend	Enumerate options, criteria, tradeoffs, then recommend	Options table → recommendation + rationale	Two-handed "weighing scales" gesture
Learning	Teach/transform knowledge	Adapt to user level, use analogies, offer follow-up quiz	Explanation → optional check-for-understanding	Open-hand explaining gesture + encouraging nod

7.2 Mandatory Manual Selection - Design Rationale

- The **ModeSelector** is always visible in the chat composer and has no default value on a new conversation; the Send control is disabled until a mode is chosen.
- A conversation stores a `default_mode` purely as a convenience pre-fill (the UI remembers the last mode the user picked so they don't have to re-click every turn) - this

is **not** an automatic classifier and every turn still carries an explicit, user-confirmed `mode` value in the request payload.

- The backend rejects any `/chat/{conversation_id}/messages` request missing a valid `mode` field (400 Bad Request: mode is required).
- There is intentionally **no intent-classification fallback mode** and no "auto" option in the UI. This removes the classifier-confidence branch that existed in the v2.0 design.
- The Query Optimization Agent (Section 5.2, step 3) is retained purely for ambiguity detection/query rewriting and must never write to the `mode` field.

7.3 Prompt Template Contract (shared skeleton)

SYSTEM:

You are Clardentity operating in {MODE_NAME} mode (selected explicitly by the user).

{MODE_SPECIFIC_INSTRUCTIONS}

{OPTIONAL_THINKING_LENS_INSTRUCTIONS}

You must ground factual claims in the provided CONTEXT block when it is relevant.

Break your answer into discrete, independently-checkable claims. Tag every claim with

a marker `<claim id="n"> ... </claim>` and, inline within it, cite supporting context with

[n] referring to the citation list. A single claim may cite more than one source.

If no supporting context exists for a claim, say so explicitly rather than inventing a source,

and leave that claim uncited so the Verification Agent marks it

Unsupported rather than

guessing at a citation.

CONTEXT:

{retrieved_chunks_with_ids}

CONVERSATION_HISTORY:

{summarized_long_term_memory}

{recent_turns}

USER:

{current_user_message}

The `<claim>` tagging allows the Verification Agent to score claims and evidence individually rather than treating the whole message as one blob (Section 9.2).

7.4 Knowing Companion - Wh-Question Query Decomposition

To make fact retrieval more precise, the Knowing Companion's Prompt Builder decomposes an incoming query using a fixed set of question lenses before running retrieval. This does

not replace the user's own phrasing - it's an internal retrieval-planning step that generates supplementary sub-queries.

Lens	Used to think about	Applied when the query concerns
Who / Whose	Ownership, attribution, belonging	People, organizations, authorship, responsibility
What	Features/attributes of non-living objects or concepts	Definitions, specs, contents of a document
Where	Location	Places, sections/pages within a document, jurisdictions
When	Time	Dates, deadlines, sequencing, versioning
Why	Reasons/causation	Rationale, justification, root cause
Which	Identification/selection among options	Disambiguating between similarly named items
How	Method/process	Procedures, mechanisms, workflows

Implementation note: `retrieval.py` runs the original query plus up to two Wh-lensed sub-queries (chosen by a lightweight keyword match against the question - e.g., a query containing "deadline" or a date word triggers the **When** lens) and merges/deduplicates the resulting top-k chunks before ranking.

7.5 Thinking Companion - Reasoning Lens (optional sub-selector)

Because auto-mode is gone, we give the user an equivalent amount of control **inside** Thinking mode instead: an optional secondary "Reasoning Lens" selector (defaults to none/unset, which uses a balanced general prompt). If set, it appends a corresponding instruction block to the Thinking Companion's system prompt. This is entirely user-driven - the system never infers a lens automatically.

Lens	Instruction injected
Analytical	Break the problem into smaller parts and address each systematically before concluding.
Critical	Evaluate the claim rationally and skeptically; identify assumptions and weak points before accepting any conclusion.
Creative	Generate unique, original angles on the problem rather than the most obvious answer.
Divergent	Generate a wide variety of distinct possibilities before narrowing down.
Convergent	Apply logic to converge on a single, well-justified answer.
Abstract	Reason about the underlying concept independent of a specific example.
Concrete	Ground the answer in specific, tangible, observable details.

Lens	Instruction injected
Associative	Draw connections between this problem and seemingly unrelated ideas that might illuminate it.
Linear	Proceed step-by-step in strict sequential order.
Non-linear	Explore connections out of sequence, following whichever thread seems most productive.
Meta-cognitive	Explicitly narrate the reasoning process itself, not just the conclusion.

Full definitions are in Appendix A.

8. Avatar Companion - Design

8.1 Purpose

A single, generic, non-identifiable on-screen avatar accompanies every conversation. It is not a chatbot personality change - it's a visual layer that reflects, in real time, (a) which cognitive mode is active and (b) how confident the current response is, so the user gets an at-a-glance emotional/contextual read alongside the text.

8.2 Rendering Approach (MVP)

- **2D, not 3D.** MVP uses a Lottie/sprite-based 2D avatar (a small, finite library of pre-authored animation clips), not a real-time 3D rig. This keeps latency and asset-authoring cost low and sidesteps uncanny-valley/likeness concerns entirely.
- The avatar is a **generic character with no resemblance to any real person** - this is a hard MVP constraint, not just a design preference (Section 8.5).
- Mouth movement while "speaking" is **amplitude-driven**, not full viseme/phoneme lip-sync: while assistant text is streaming (or TTS audio is playing, if the user requested audio output), the avatar cycles a simple "talking" mouth-loop animation; it stops and returns to a closed-mouth idle frame when streaming ends. Full phoneme-accurate lip-sync is a Post-MVP stretch (Section 17).

8.3 State Machine

State	Trigger	Animation
Idle	No active turn	Gentle breathing/blink loop
Listening	User is typing or recording audio	Head-tilt-toward-user pose
Thinking	Request sent, awaiting/streaming generation	Mode-specific gesture loop (Section 7.1 table)

State	Trigger	Animation
Speaking	Response text streaming / TTS audio playing	Talking mouth-loop overlaid on the mode gesture
Reacting	Final confidence band resolved	One-shot expression transition (Section 8.4), then return to Idle

8.4 Expression & Gesture Mapping

Two independent signals combine to produce the `avatar_cue` attached to each finished message:

1. **Gesture** - determined by `mode_used` (table in Section 7.1): Knowing → presenting, Thinking → chin-stroke, Decision → weighing-scales, Learning → open-hand explaining.
2. **Expression** - determined by the confidence band, then overridden if the Verification Agent raised a distortion flag:

Confidence band	Base expression
Likely Fact	Confident / affirming (slight nod)
Plausible	Neutral / thoughtful
Needs Verification	Cautious (slight head-tilt, raised eyebrow)
<i>(any band, if a wishful/magical-thinking flag was raised - Section 9.4)</i>	Concerned / cautionary - overrides the base expression regardless of the numeric band

`avatar_cue = {expression, gesture}` is computed server-side once the Confidence Scoring step (Section 9.3) completes, and is included in the final SSE payload and persisted on the message (Section 10).

8.5 Constraints

- No user-uploaded avatar images, no face-swap, no attempt to depict a real or user-resembling person, in MVP or otherwise planned for this product line - the avatar stays generic by design.
- Avatar rendering is entirely client-side (the backend only emits the small `{expression, gesture}` enum pair); it must add no perceptible latency to message streaming.
- Avatar must degrade gracefully (static idle frame) if animation assets fail to load - never block the chat UI.

9. Validation Engine, Confidence Scoring & Evidence Transparency

9.1 Pipeline (Reflection Agent + Verification Agent)

1. **Candidate generation** - OpenAI Responses API produces the draft answer with `<claim>` tags and inline `[n]` citation markers (Section 7.3).
2. **Reflection Agent** - critiques the draft for internal consistency, completeness, and fit with the selected mode's expected output shape; may request one revision pass from the generator before handing off.
3. **Verification Agent**, for **each** `<claim>`: a. Identifies every citation marker referenced inside that claim. b. For **each** cited piece of evidence individually: runs a semantic-similarity check plus an LLM entailment check (*does this specific evidence support this specific claim? full / partial / no*) and records a `support_score` and `relevance_score` for that claim–evidence pair. c. If a claim has zero citations, it is recorded with an empty evidence list and an `Unsupported` label - never silently dropped. d. Runs the claim's text against the cognitive-distortion checklist (Section 9.4).
4. **Confidence Scoring** aggregates the per-claim, per-evidence scores into a per-claim score, then into one message-level band (Section 9.3).
5. All of the above - per-claim text, per-evidence source, per-evidence score, entailment label, and any distortion flag - is stored (Section 10) and rendered in full (Section 9.5). Nothing is collapsed into a single opaque number before it reaches storage or the UI.

9.2 Per-Claim, Per-Evidence Scoring (replaces the single aggregate-only score)

Where v2.0 computed one

`entailment_score / citation_coverage / retrieval_relevance` triple for the whole message, v3.0 computes these **per claim, and per evidence item within each claim**:

- **Per evidence item** (one row per claim–citation pair): `entailment_label` (`full` / `partial` / `none`), `support_score` (0–1, how strongly this specific source backs this specific claim), `relevance_score` (0–1, how topically relevant the retrieved chunk was to the query in general).
- **Per claim**: `claim_score = 100 * (0.7 * best_evidence.support_score + 0.3 * best_evidence.relevance_score)` , using whichever evidence item best supports the claim. A claim with no evidence gets `claim_score = 0` and label `Unsupported` .
- **Per message** (rollup, Section 9.3): combines all claim scores plus overall citation coverage.

This lets the UI show, for any single sentence in a response, exactly which source(s) were checked, how well each one held up, and why the sentence got the score it did - satisfying FR16/FR17.

9.3 Scoring Formula

Message-level aggregate (unchanged in spirit from v2.0, now explicitly built from the per-claim numbers in 9.2 rather than one whole-message pass):

```

citation_coverage    = (# claims with at least one evidence item) / (#
total claims)
mean_claim_score     = average(claim_score) across all claims, in [0,100]
mean_relevance       = average(relevance_score) across all evidence items
actually used

message_score = 0.6 * mean_claim_score + 0.25 * (100 * citation_coverage)
+ 0.15 * (100 * mean_relevance)

```

Banding: `>=90 Likely Fact`, `70–89 Plausible`, `<70 Needs Verification`. All weights are configurable via `/admin`.

Distortion penalty: if any claim in the message was flagged by the cognitive-distortion screen (Section 9.4), subtract 15 points from `message_score` (floor 0) and cap the resulting band at `Plausible` even if the numeric score would otherwise round up to `Likely Fact` - a response that reasons via wishful or magical thinking should never present as fully trustworthy, regardless of how well-cited its other claims are.

9.4 Cognitive Distortion Screening (Wishful & Magical Thinking anti-patterns)

The Verification Agent screens each claim's reasoning - not just its factual grounding - against two named distortion patterns, and surfaces *why* a flagged claim is a problem using the same purpose/result framing an educator would use, so the flag is explainable to the user rather than a bare label:

Pattern	Definition	What the Verification Agent looks for
Wishful thinking	Favoring an answer because it's pleasing or convenient to imagine, rather than because it follows from proven facts or accepted evidence.	A claim's conclusion is stated with confidence while its cited (or absent) evidence only weakly or not at all supports it - i.e., the claim reaches beyond what its evidence can carry.
Magical thinking	Asserting a causal or correlational link between an action/event and an outcome with no plausible causal mechanism connecting them.	A claim asserts "X caused/leads to Y" where the retrieved evidence shows X and Y co-occurring (or neither) but supplies no mechanism connecting them.

When either pattern is detected on a claim, the API response includes a `distortion_flag` on that claim (`wishful_thinking` / `magical_thinking`) with a one-line, plain-language explanation (e.g., *"This conclusion goes further than the cited source actually supports."*), the message-level penalty in 9.3 applies, and the avatar's expression overrides to "concerned" (Section 8.4) regardless of the numeric band.

Full source definitions, including the purpose/advantage/disadvantage framing these were drawn from, are in Appendix A.

9.5 Evidence Transparency in Responses (UI requirement - FR17)

Every assistant message renders:

1. **Inline markers** - [1] , [2] , ... next to the specific sentence/claim they support, exactly as generated.
2. **An "Evidence" panel** (expanded by default for Needs Verification messages, collapsible otherwise) listing, per claim, in order:
 - The claim text
 - Each cited source with its excerpt, document name, page/chunk reference, and its individual support_score / relevance_score
 - Claims with no citation explicitly rendered as **"No supporting evidence found for this statement"** rather than being omitted
 - Any distortion_flag and its plain-language explanation
3. **The message-level confidence badge**, which links/scrolls to the Evidence panel when clicked.

This is a hard requirement, not a "nice to have" - a response is not considered spec-compliant if it shows only an aggregate badge without the underlying per-claim breakdown.

10. Data Model (PostgreSQL)

```
-- Users & Auth
users (
  id UUID PK,
  email TEXT UNIQUE NOT NULL,
  password_hash TEXT,           -- null if OAuth-only
  oauth_provider TEXT,
  oauth_subject TEXT,
  display_name TEXT,
  created_at TIMESTAMPTZ DEFAULT now()
)

-- Workspaces
workspaces (
  id UUID PK,
  owner_id UUID FK -> users.id,
  name TEXT NOT NULL,
  created_at TIMESTAMPTZ DEFAULT now()
)
```

```

workspace_members (
  workspace_id UUID FK -> workspaces.id,
  user_id UUID FK -> users.id,
  role TEXT CHECK (role IN ('owner', 'member')),
  PRIMARY KEY (workspace_id, user_id)
)

-- Conversations & Messages
conversations (
  id UUID PK,
  workspace_id UUID FK -> workspaces.id,
  title TEXT,
  default_mode TEXT CHECK (default_mode IN
('knowing', 'thinking', 'decision', 'learning')),
  -- convenience pre-fill only; every message still carries its own
explicit `mode` (below)
  created_at TIMESTAMPTZ DEFAULT now()
)

messages (
  id UUID PK,
  conversation_id UUID FK -> conversations.id,
  role TEXT CHECK (role IN ('user', 'assistant', 'system')),
  content TEXT,
  mode_used TEXT CHECK (mode_used IN
('knowing', 'thinking', 'decision', 'learning')) NOT NULL,
  reasoning_lens TEXT,          -- optional, Thinking mode only (Section
7.5), nullable
  confidence_score NUMERIC,      -- rolled-up message_score (Section 9.3)
  confidence_band TEXT,
  distortion_penalty_applied BOOLEAN DEFAULT false,
  avatar_expression TEXT,        -- Section 8.4
  avatar_gesture TEXT,          -- Section 8.4
  token_usage JSONB,
  created_at TIMESTAMPTZ DEFAULT now()
)

-- Per-claim breakdown
message_claims (
  id UUID PK,
  message_id UUID FK -> messages.id,
  claim_index INTEGER,
  claim_text TEXT,
  claim_score NUMERIC,          -- Section 9.2
  entailment_label TEXT CHECK (entailment_label IN
('full', 'partial', 'none', 'unsupported')),
  distortion_flag TEXT CHECK (distortion_flag IN
('wishful_thinking', 'magical_thinking', NULL)),
  distortion_explanation TEXT,
  created_at TIMESTAMPTZ DEFAULT now()
)

```



```

)

-- Per-claim, per-evidence breakdown (replaces a single citation-only
view)
claim_evidence (
    id UUID PK,
    claim_id UUID FK -> message_claims.id,
    citation_id UUID FK -> citations.id,    -- nullable if evidence is web-
sourced without a stored document
    support_score NUMERIC,                  -- 0-1, Section 9.2
    relevance_score NUMERIC,                -- 0-1, Section 9.2
    entailment_label TEXT CHECK (entailment_label IN
('full', 'partial', 'none')),
    source_excerpt TEXT,
    created_at TIMESTAMPTZ DEFAULT now()
)

citations (
    id UUID PK,
    message_id UUID FK -> messages.id,
    document_id UUID FK -> documents.id,
    chunk_id UUID FK -> document_chunks.id,
    source_type TEXT CHECK (source_type IN ('document', 'web')) DEFAULT
'document',
    marker INTEGER,                        -- corresponds to [n] in message content
    relevance_score NUMERIC
)

-- Documents & RAG
documents (
    id UUID PK,
    workspace_id UUID FK -> workspaces.id,
    filename TEXT,
    file_type TEXT,
    storage_path TEXT,                    -- S3-compatible object key
    status TEXT CHECK (status IN
('uploading', 'processing', 'processed', 'failed')),
    uploaded_by UUID FK -> users.id,
    created_at TIMESTAMPTZ DEFAULT now()
)

document_chunks (
    id UUID PK,
    document_id UUID FK -> documents.id,
    chunk_index INTEGER,
    content TEXT,
    embedding VECTOR(1536),              -- pgvector; dim matches chosen OpenAI
embedding model
    page_number INTEGER,
    created_at TIMESTAMPTZ DEFAULT now()
)

```

```

)

-- Long-term memory
conversation_memory (
  id UUID PK,
  conversation_id UUID FK -> conversations.id,
  summary TEXT, -- rolling summarized memory
  updated_at TIMESTAMPTZ DEFAULT now()
)

-- Audio
audio_transcripts (
  id UUID PK,
  message_id UUID FK -> messages.id,
  storage_path TEXT,
  transcript TEXT,
  duration_seconds NUMERIC
)

-- Admin / Config
admin_settings (
  key TEXT PK,
  value JSONB,
  updated_at TIMESTAMPTZ DEFAULT now()
)

```

Indexes: document_chunks.embedding via ivfflat (pgvector) index;
 messages.conversation_id; documents.workspace_id; message_claims.message_id;
 claim_evidence.claim_id; full-text (tsvector) index on messages.content for FR12
 search.

11. API Specification

Base path: /api/v1

11.1 /auth

Method	Path	Body	Response
POST	/auth/register	{email, password}	{user, access_token, refresh_token}
POST	/auth/login	{email, password}	{access_token, refresh_token}
POST	/auth/oauth/google	{id_token}	{access_token, refresh_token}
POST	/auth/refresh	{refresh_token}	{access_token}

11.2 /chat

Method	Path	Body	Response
POST	/chat/{conversation_id}/messages	{content, attachments[], mode, reasoning_lens?} - mode is required; request is rejected with 400 if absent	SSE stream: {delta} chunks → final {message, claims[], confidence, avatar_cue}
GET	/chat/{conversation_id}/messages	-	paginated message list, each with nested claims[]
POST	/chat/conversations	{workspace_id, title?}	{conversation_id}
DELETE	/chat/conversations/{id}	-	204

Final SSE payload shape (expanded from v2.0):

```
{
  "message": { "id": "...", "content": "...", "mode_used": "decision" },
  "claims": [
    {
      "claim_text": "...",
      "claim_score": 82,
      "entailment_label": "partial",
      "distortion_flag": null,
      "evidence": [
        { "citation_marker": 1, "document": "VendorA_contract.pdf",
"excerpt": "...", "support_score": 0.8, "relevance_score": 0.9 }
      ]
    }
  ],
  "confidence": { "score": 78, "band": "Plausible" },
  "avatar_cue": { "expression": "thoughtful", "gesture": "weighing_scales"
}
}
```

11.3 /documents

Method	Path	Body	Response
POST	/documents/upload	multipart {workspace_id,	{document_id, status:

Method	Path	Body	Response
		file}	"processing"
GET	/documents/{id}	-	{document, status, chunk_count}
DELETE	/documents/{id}	-	204
GET	/documents?workspace_id=	-	list of documents

11.4 /audio

Method	Path	Body	Response
POST	/audio/transcribe	multipart {file}	{transcript, duration_seconds}
POST	/audio/tts	{text, voice?}	audio stream (mp3)

11.5 /memory

Method	Path	Response
GET	/memory/{conversation_id}	{summary, last_updated}
POST	/memory/{conversation_id}/rebuild	forces summary regeneration

11.6 /validation

Method	Path	Response
GET	/validation/{message_id}	{score, band, claims: [{claim_text, claim_score, entailment_label, distortion_flag, evidence: [{citation_id, support_score, relevance_score, entailment_label, excerpt}]}]}

11.7 /history

Method	Path	Response
GET	/history/search?workspace_id=&q=	ranked matching messages

11.8 /admin

Method	Path	Body	Response
GET	/admin/settings	-	all config key/values (includes scoring weights, avatar gesture map)
PUT	/admin/settings	{key, value}	updated setting

All authenticated endpoints require `Authorization: Bearer <access_token>`.

12. Multimodal Handling Detail

12.1 Audio

- Client records via browser `MediaRecorder` or uploads a file.
- Backend forwards to OpenAI's speech-to-text endpoint; transcript stored in `audio_transcripts` and inserted into the chat pipeline as the user turn content.
- TTS: assistant responses can optionally be rendered to audio via OpenAI's TTS endpoint on-demand (not generated by default, to save cost); when TTS is playing, the avatar's Speaking state (Section 8.3) syncs its talking-loop to audio playback rather than to text-streaming.

12.2 Images

- Uploaded images are stored in S3-compatible storage; a signed reference (or base64 for small images) is passed to the OpenAI Responses API vision input alongside the text turn.
- Images are **not** chunked/embedded for RAG in MVP (deferred) - they are treated as direct multimodal context for that turn only, unless the user explicitly asks to "save this image to the workspace," in which case a caption/description is generated and embedded as a searchable chunk.

12.3 Documents

- Supported: PDF, DOCX, TXT.
- Pipeline: upload → text extraction (per-type parser) → chunking (~500–800 tokens per chunk, ~15% overlap) → embedding via OpenAI embeddings → store in `document_chunks` with `pgvector`.
- Processing status polled by client (`processing` → `processed` / `failed`) or pushed via WebSocket.

12.4 Avatar Output

- The avatar is the platform's one non-text-input, always-on **output** modality: every assistant turn, regardless of input type, resolves to an `{expression, gesture}` pair rendered client-side (Section 8).
-

13. Conversation Memory Design

- **Short-term:** last N turns (configurable, default 20) included verbatim in the prompt.
 - **Long-term:** a rolling summary (`conversation_memory.summary`) regenerated via a background Celery task every M turns (default 15) or when short-term window would exceed token budget. Summary captures key facts, decisions, and user preferences stated in the conversation.
 - Both are merged into the Prompt Builder's `CONVERSATION_HISTORY` block (Section 7.3).
-

14. Non-Functional Requirements

Category	Requirement
Latency	P95 < 6s for text-only responses (excluding streaming display time); P95 < 15s for document-grounded responses with per-claim validation
Avatar latency	Avatar cue computation adds negligible (<50ms) server-side overhead; all animation rendering is client-side and must never block message streaming
Scalability	Support 100 concurrent users on MVP infra without degradation
Availability	99.5% target for MVP (single-region)
Security	TLS in transit; AES-256 at rest for S3 and DB volumes; JWT with short-lived access tokens + refresh rotation
Observability	Structured logging, request tracing (correlation IDs), error rate + latency dashboards
Resilience	Retry with exponential backoff on OpenAI API calls (max 3 attempts); circuit breaker on repeated failures
Audit	All admin config changes logged with actor + timestamp + diff
Cost control	Token usage logged per message; configurable per-workspace monthly token cap (soft warning, not hard block in MVP); per-claim/per-evidence scoring adds extra LLM calls - track and cap these separately in admin

15. Security Requirements

- Passwords hashed with bcrypt/argon2; never logged.
- JWT access tokens short-lived (15 min); refresh tokens rotated and revocable.
- File uploads scanned for type/size before processing; reject executable/script content types.
- Workspace-level access control enforced at the query layer (every document/conversation query filtered by workspace membership).
- Secrets (OpenAI API key, DB credentials) via environment variables / secrets manager - never committed to repo.
- Rate limiting on `/auth` and `/chat` endpoints to mitigate abuse.
- Avatar assets are static, bundled, generic art assets - no user-supplied imagery is ever rendered as or blended into the avatar.

16. Repository Structure (for Cursor scaffolding)

```

clardentity/
├── frontend/                                # Next.js app
│   ├── app/
│   │   ├── (auth)/login/page.tsx
│   │   ├── (auth)/register/page.tsx
│   │   ├── workspace/[id]/page.tsx
│   │   ├── chat/[conversationId]/page.tsx
│   │   └── admin/page.tsx
│   ├── components/
│   │   ├── chat/ (MessageList, MessageInput, ConfidenceBadge,
EvidencePanel, CitationPopover, ModeSelector, ReasoningLensSelector)
│   │   ├── avatar/ (AvatarPanel, avatar state machine +
gesture/expression clip loader)
│   │   ├── upload/ (DocumentUploader, AudioRecorder)
│   │   └── workspace/ (WorkspaceSidebar, ProjectList)
│   ├── lib/ (apiClient.ts, auth.ts, sse.ts, avatarCues.ts)
│   ├── public/avatar/ (generic avatar animation clips: idle, listening,
gestures per mode, expressions per band)
│   └── styles/
└── backend/                                # FastAPI app
    ├── app/
    │   ├── main.py
    │   └── api/
    │       ├── auth.py
    │       ├── chat.py
    │       ├── documents.py
    │       ├── audio.py
    │       ├── memory.py
    │       ├── validation.py
    │       └── admin.py

```

```

├── core/
│   ├── config.py
│   ├── security.py (JWT, password hashing)
│   └── celery_app.py
├── models/ (SQLAlchemy models mirroring Section 10, incl.
message_claims, claim_evidence)
├── schemas/ (Pydantic request/response models)
├── services/
│   ├── query_optimizer.py # Section 5.2 step 3 -
ambiguity/rewrite only
│   ├── router.py # Cognitive Router - validates
user-selected mode (Section 7.2)
│   ├── retrieval.py # pgvector similarity search +
Wh-lens sub-queries (Section 7.4)
│   ├── prompt_builder.py # incl. reasoning-lens injection
(Section 7.5)
│   ├── openai_client.py
│   ├── reflection_agent.py # Section 9.1
│   └── verification_agent.py # per-claim/per-evidence scoring
+ distortion screen (Section 9)
│   ├── confidence_scoring.py # Section 9.3 rollup formula
│   ├── avatar_cue_service.py # Section 8.4 mapping logic
│   ├── memory_service.py
│   ├── document_ingestion.py
│   └── audio_service.py
├── workers/ (Celery tasks: ingest_document, rebuild_memory,
transcribe_audio)
├── db/ (session, migrations via Alembic)
├── tests/
└── requirements.txt

├── infra/
│   ├── docker-compose.yml # postgres+pgvector, redis, backend,
frontend
│   └── .env.example
└── docs/
    ├── this SRS
    ├── clardentity_flow_diagram.png
    └── clardentity sequence diagram.png

```

17. Post-MVP Roadmap (explicitly out of MVP scope)

1. **Video ingestion** - upload + frame-sampling + transcript extraction, fed into the same RAG pipeline; video reasoning mode.

2. **3D avatar with full viseme/phoneme lip-sync**, replacing the MVP's amplitude-driven 2D mouth-loop.
 3. Collaborative multi-user live sessions (shared cursors, presence).
 4. Enterprise SSO (SAML/SCIM) and fine-grained RBAC.
 5. Analytics dashboards (usage, mode distribution, confidence trends over time).
 6. Web-search-augmented validation as a first-class toggle (MVP validation is document-grounded only, with web-search cross-check as an optional stretch item if time allows).
-

18. Cursor Build Order (phased, file-level)

Phase 1 - Scaffold

- Initialize `frontend/` (Next.js + Tailwind) and `backend/` (FastAPI) per Section 16 structure.
- `docker-compose.yml` for Postgres(+pgvector), Redis.
- `.env.example` with `OPENAI_API_KEY`, `DATABASE_URL`, `REDIS_URL`, `S3_*`, `JWT_SECRET`.

Phase 2 - Auth & Database

- SQLAlchemy models + Alembic migrations for all tables in Section 10, including `message_claims` and `claim_evidence`.
- `/auth` endpoints, JWT issuance/refresh, password hashing.
- Frontend login/register pages + auth context.

Phase 3 - Chat Core + OpenAI Integration

- `/chat` endpoints with SSE streaming, `mode` required on every request (reject if missing).
- `openai_client.py` wrapping the Responses API.
- Basic `MessageList/MessageInput` + mandatory `ModeSelector` UI - get end-to-end text chat working with real mode selection before adding retrieval/validation.

Phase 4 - Document Ingestion + RAG

- `/documents` upload endpoint + S3-compatible storage integration.
- `document_ingestion.py`: parsers (PDF/DOCX/TXT) → chunking → embeddings → pgvector insert.
- `retrieval.py`: top-k similarity search + Wh-lens sub-query expansion for Knowing mode (Section 7.4).
- Wire retrieval into Prompt Builder; add basic citation rendering in UI.

Phase 5 - Memory & Cognitive Router

- `memory_service.py` : short-term window + rolling summary Celery task.
- `query_optimizer.py` : ambiguity detection/query rewrite only (no mode logic).
- `router.py` : validates selected mode, loads mode-specific prompt template (Section 7.3), applies optional Reasoning Lens for Thinking mode (Section 7.5).
- ModeSelector + ReasoningLensSelector UI components.

Phase 6 - Validation Engine (Reflection + Verification Agents)

- `reflection_agent.py` : draft critique pass.
- `verification_agent.py` : `<claim>` extraction, per-claim per-evidence entailment checks, distortion screening (Section 9.4).
- `confidence_scoring.py` : per-claim scores → message rollup formula (Section 9.3), distortion penalty.
- `/validation/{message_id}` endpoint; `EvidencePanel` UI component rendering the full per-claim/per-evidence breakdown (Section 9.5) - not just a badge.

Phase 7 - Avatar Companion

- `avatar_cue_service.py` : mode → gesture, confidence band (+ distortion override) → expression mapping (Section 8.4).
- `AvatarPanel` frontend component + state machine (idle/listening/thinking/speaking/reacting, Section 8.3) using bundled generic 2D animation clips.
- Wire `avatar_cue` into the final SSE payload and message persistence.

Phase 8 - Multimodal, History, Export, Admin, Polish

- `/audio` transcription + optional TTS; `AudioRecorder` component; sync avatar Speaking state to TTS playback.
- Image input wiring into chat (vision payload).
- `/history` search endpoint + search UI.
- Conversation export (Markdown/PDF) including per-claim evidence.
- `/admin` settings endpoint + admin UI (model/temperature/top-k/scoring weights/avatar gesture map/feature flags).
- End-to-end testing, load testing to 100 concurrent users, deployment scripts.

19. Testing Requirements

Layer	Approach
Unit	Mode-selection enforcement (reject missing mode); per-claim scoring formula edge cases (zero-evidence claim, all-partial evidence); distortion-

Layer	Approach
	screen true/false positive cases; chunking logic; avatar cue mapping table (every mode × band × distortion combination)
Integration	Document upload → embed → retrieve → per-claim cite round trip; auth flow; SSE streaming with <code>claims[]</code> + <code>avatar_cue</code> in final payload
E2E	Full user journey: signup → create workspace → upload doc → explicitly select each of the 4 modes → confirm per-claim evidence panel + confidence + avatar reaction render correctly → export conversation
Load	100 concurrent simulated users hitting <code>/chat</code> - verify P95 latency targets (Section 14), including the added per-claim/per-evidence validation calls
Security	Auth bypass attempts, workspace isolation (user A cannot see workspace B's documents), file upload type/size validation

20. Acceptance Criteria (MVP Done Definition)

- ☐ User can register, log in, and create a workspace.
- ☐ User can upload PDF/DOCX/TXT/image documents; ingestion completes and is queryable.
- ☐ User can converse in text, with audio input transcribed correctly.
- ☐ **A message cannot be sent without the user first explicitly selecting one of the four cognitive modes; there is no auto/default mode.**
- ☐ **Every assistant response shows a per-claim evidence breakdown - not just an aggregate badge - with each claim's own score and each evidence item's own score, and unsupported claims explicitly labeled as such.**
- ☐ **Every assistant response is accompanied by an avatar whose gesture matches the selected mode and whose expression matches the resulting confidence band (with a distinct "concerned" override when a distortion flag is raised).**
- ☐ Conversation history persists, is searchable, and is exportable with full evidence detail intact.
- ☐ Admin can adjust model parameters and scoring weights and see them take effect.
- ☐ System meets latency and concurrency targets under a basic load test, inclusive of the per-claim validation overhead.

21. Out of Scope (restated)

Video ingestion and real-time video reasoning, collaborative live editing, enterprise SSO, fine-grained RBAC, analytics dashboards, and any 3D/photorealistic or user-likeness avatar - all explicitly deferred to Post-MVP (Section 17).

Appendix A. Cognitive Taxonomy Reference

This appendix consolidates the taxonomies referenced in Sections 7.4, 7.5, and 9.4 in one place for engineers implementing the prompt templates and the Verification Agent.

A.1 Wh-Questions (Knowing Companion query decomposition - Section 7.4)

Question	Used to think about
Who and Whose	The belongings and ownership of living beings
What	The features of non-living objects and things
Where	Location
When	Time
Why	Reasons
Which	Identification and selection
How	Functioning methods and processes of things

A.2 Types of Thinking (Thinking Companion reasoning lenses - Section 7.5)

Type	Definition
Abstract thinking	The ability to think about complex concepts and ideas without being tied to specific examples, objects, people, or situations.
Analytical thinking	A systematic way of solving problems and making decisions by breaking down information into smaller parts.
Associative thinking	A way of thinking that involves connecting seemingly unrelated ideas and thoughts.
Concrete thinking	A literal way of thinking that focuses on the physical world and what you can see, hear, feel, and experience in the present.
Creative thinking	The ability to come up with unique, original solutions.
Critical thinking	The process of analyzing information to make a judgment using rational, unbiased, and skeptical evaluation.
Divergent thinking	A creative thought process that involves generating a variety of ideas in response to an open-ended problem or question.
Convergent thinking	A cognitive process that involves analyzing information and applying logic to find a single, correct solution to a problem.

Type	Definition
Linear thinking	A systematic, analytical, and logical way of thinking that follows a step-by-step process.
Non-linear thinking	A way of thinking that involves making connections between seemingly unrelated ideas, rather than thinking in a sequential or linear manner.
Meta-cognition	The process of thinking about thinking, or being aware of and controlling one's own cognitive processes.

A.3 Thinking Styles (informs Decision/Thinking companion framing)

Style	Definition
Synthesist thinking	A creative and curious style of thinking that involves considering multiple ideas, views, and possibilities.
Idealist thinking	A way of thinking characterized by a focus on making the world a better place, developing personal values, and envisioning an ideal world.
Pragmatist thinking	A practical way of using experience to solve problems and make decisions.
Analyst thinking	A systematic way of solving problems and making decisions by breaking down information into smaller parts.
Problem-solver thinking	The ability to use facts, knowledge, and data to identify, analyze, and resolve problems.

A.4 Wishful Thinking vs. Magical Thinking (Verification Agent distortion screen - Section 9.4)

Wishful thinking

- **Definition:** Asking questions about one's own or others' thoughts in a way that seeks answers which feel pleasing or interesting to imagine - or reaches for a random guess - rather than relying on proven facts and non-facts backed by scientifically and legally accepted evidence.
- **Purpose:** To find answers that feel pleasing and interesting, rather than grounding the answer in evidence.
- **Advantage:** Temporarily (and, if unchecked, permanently) blinds the thinker to the proven facts and non-facts in front of them - i.e., it feels good in the moment.
- **Disadvantage:** Leads to several risks in achieving sustainable progress in individual and collective welfare and wellbeing.

Magical thinking

- **Definition:** A cognitive distortion or inaccurate thinking pattern (belief) about the result of an event that occurs without any plausible causal link to one's ideas, thoughts, actions, inactions, words, or use of sign/symbolic language.
- **Purpose:** To deal with fear, low confidence, and to maintain hope of a positive outcome from one's own actions and inactions.
- **Advantage:** Can help children roughly up to age 10–15, fading as they mature - how much it fades depends on their education and upbringing.
- **Disadvantage:** If it doesn't fade with education/upbringing into adulthood, it can correlate with OCD or GAD; it encourages and supports ignorance and delusion in people of any age or gender; and it creates opportunities for fraud, criminal behavior, and exploitation of others.

The Verification Agent uses the **definition** column of both patterns as its detection criterion (Section 9.4) and surfaces the **purpose** framing in the plain-language explanation shown to the user, so a flagged claim reads as an explanation of a reasoning pattern rather than an accusation.